

OPTIMIZING CPU INFERENCE OF A PROJECTIVE GEOMETRIC ALGEBRA TRANSFORMER

Anastasija Tortevska*, Glenn Schönbächler*, Kristijan Zafirovski*, Mukhammadali Sayfiddinov*

Department of Computer Science
ETH Zurich, Switzerland

ABSTRACT

The Geometric Algebra Transformer (GATr) is an $E(3)$ -equivariant transformer whose layers act on multivectors of the 16-dimensional projective geometric algebra (PGA) instead of plain vectors. Its reference implementation, EzGATr, is written for clarity in PyTorch and is dominated by many small, irregular tensor contractions that map poorly onto a general tensor runtime, making single-core CPU inference slow. We close this gap by hand-writing dedicated C++ kernels for its six performance-critical operators: the geometric product, the equivariant join, the equivariant linear map, the equivariant RMS normalization, the scalar-gated GELU, and the geometric inner-product and distance-aware attention. We optimize each through a common four-stage ladder that exploits algebraic sparsity and identities, removes redundant work and memory traffic, and finally adds explicit AVX-512 and AVX2 vectorization with FMA. On an Intel Core i7-1165G7 (Tiger Lake) at one thread, we obtain per-kernel runtime speedups of up to $346\times$, a $38\times$ end-to-end speedup over a naive C++ baseline on the full model at small sizes that trends toward roughly $13\times$ as the sequence grows, and per-operator speedups of 2.2 to $69.5\times$ over the PyTorch reference. The two compute-bound kernels, geometric attention and the equivariant linear map, reach 71% and 60% of single-core peak, while the memory-bound kernels roll off to the bandwidth roof as the working set grows, exactly as the cost model predicts.

1. INTRODUCTION

Motivation. Equivariant neural networks encode the symmetries of 3D space (rotations, translations, reflections) directly into the architecture, so a rotated input produces a correspondingly rotated output by construction rather than by data augmentation. This sharply improves sample efficiency on geometric tasks such as molecular property prediction, robotics, and physical simulation. The Geometric Algebra Transformer (GATr) [1] is one such architecture. It represents every token as a multivector of the 16-

dimensional projective geometric algebra (PGA), so points, lines, planes, and translations share one algebraic object, and every layer is an equivariant linear or bilinear map on multivectors. This equivariance is not free. Each layer replaces a standard transformer’s dense matrix multiplications with many small contractions that follow a fixed, sparse pattern of ± 1 signs. EzGATr [2] expresses these as PyTorch `einsum` calls, which are readable and correct but slow on a CPU. The contractions are tiny, with an inner dimension of only 16 and a few percent nonzeros, yet the framework dispatches one generic kernel per call with no knowledge of the fixed pattern. Inference latency matters for the settings GATr targets, such as robotics control loops and interactive simulation, which do not always have a GPU.

Related work. GATr [1] and EzGATr [2] define the algorithm we optimize, with the EzGATr PyTorch operators as our correctness reference. Our flash attention core follows the online-softmax tiling of FlashAttention [3], specialized here to the small per-head geometric feature width. We found no prior work that hand-optimizes PGA multivector kernels for CPU, which is the gap this project fills.

Contribution. We present a CPU-inference-optimized re-implementation of the six dominant EzGATr operators as C++ kernels with explicit AVX-512 and AVX2 vectorization, organized as a reproducible four-stage optimization ladder from $v0$ to $v3$. We make three contributions. First, we derive a precise FLOP and memory-traffic cost model for each operator (Section 2). Second, we present the optimizations themselves, namely sparsity exploitation, redundant-work and memory-traffic elimination, and explicit AVX-512 and AVX2 vectorization with FMA, including a fused, cache-resident flash-SDPA core that assembles the geometric attention features in-place (Section 3). Third, we give a roofline-grounded experimental study that separates the algorithmic gains from the locality and SIMD gains and compares our hand-written vectorization against the compiler’s auto-vectorizer and across two compilers (Section 4).

Authors are listed in alphabetical order;

2. BACKGROUND ON THE ALGORITHM/APPLICATION

A GATr block mirrors a standard transformer block but replaces its dot products and dense linear layers with geometric-algebra operations on multivectors.

Multivectors and the layer structure. A token is a multivector $x \in \mathbb{R}^{16}$, indexed by the 16 *blades* of the 3D PGA. A hidden activation is a tensor of $M = BT$ such multivectors with C channels each, i.e. shape $(B, T, C, 16)$ for batch B and sequence length T . Like a standard transformer, a GATr block has two residual sub-blocks. The first applies an equivariant normalization, an equivariant linear map, and the geometric attention. The second applies another normalization and linear map, a bilinear block where the geometric product and the equivariant join combine multivectors channel-wise, with the channel mixing supplied by the linear maps that form their operands, and a scalar-gated GELU. These sub-blocks are built from the six operators we optimize, described next.

Geometric product and equivariant join. Both are bilinear maps that combine two multivectors $x, y \in \mathbb{R}^{16}$ into a third, $o_i = \sum_{j,k} \Gamma_{ijk} x_j y_k$, where Γ is a fixed, sparse structure tensor that differs between the two. A dense evaluation costs $16^3 = 4096$ multiply-adds (MACs) per multivector, but Γ has only 192 nonzeros ($\approx 5\%$) for the geometric product and 81 ($\approx 2\%$) for the join, and every nonzero is ± 1 . The join additionally scales its output by the e_{123} component (blade 14) of a data-dependent reference multivector formed as the mean over tokens, not by a learned parameter.

Equivariant linear map. The most general equivariant linear map on PGA multivectors is spanned by 9 basis maps. `equi_linear` mixes C_{in} into C_{out} channels by contracting each (o, i) channel pair against these 9 coefficients applied blade-wise. A dense 16×16 blade matrix per channel pair would cost 256 MACs, but the equivariant structure leaves only 24 nonzero couplings (16 diagonal and 8 off-diagonal), so the map costs $24 C_{\text{in}} C_{\text{out}}$ MACs per token. It is the only kernel whose cost is quadratic in the channel count C , so it dominates the non-attention compute.

Equivariant RMS norm and scalar-gated GELU. The RMS norm applies a shared scale $1/\sqrt{\bar{r}}$, where we define $\bar{r} = \frac{1}{C} \sum_c \langle x_c, x_c \rangle$ as the mean invariant inner product over all C channels. The inner product $\langle x, x \rangle$ sums the squares of only the 8 metric-active blades 0, 2, 3, 4, 8, 9, 10, 14. The remaining 8 blades involve the null generator e_0 and carry zero norm under the PGA metric. All C multivectors are then rescaled by $w_c/\sqrt{\bar{r}}$, where w_c is an optional per-channel learnable weight. The scalar-gated GELU forms a gate $g = \text{GELU}(x_0)$ from the scalar blade and broadcasts $x \cdot g$ across all 16 blades, paying one $\tanh$ per multivector. Both do only a few flops per blade over the full $16C$ activation, so they are memory-bound.

Geometric attention. Attention scores combine two equivariant similarities of the query and key multivectors. The inner-product term (IPA) compares 7 metric blades, and the distance-aware term (DAA) is built from the tri-vector blades $\{11, 12, 13, 14\}$, which encode points. The two form a per-head feature of width $12C$, 7 from IPA and 5 from DAA per channel, on which a standard scaled-dot-product attention (SDPA) with softmax runs while the values keep the full $16C$ width. Its $T \times T$ scores make attention the only $O(T^2)$ operator, so it dominates at long sequences.

Cost measure. We count useful flops W , taking each multiply-add as 2 flop and excluding structural zeros. Performance is $P = W/t$ and intensity $I = W/Q$, with compulsory traffic Q to the L1 boundary that reads each input once, writes each output once, and amortizes weights over M tokens. Per-token work and intensity at C channels are:

kernel	W [flop/token]	$I = W/Q$	$I _{C=16}$
equi. linear	$48 C^2$	$0.375 C$	6.0
geom. product	$384 C$	2.0	2.0
equi. join	$162 C$	0.63	0.63
gated GELU	$\approx 16 C (+ \tanh)$	<1	mem.
RMS norm	$\approx 31 C$	<1	mem.

Attention is the only $O(T^2)$ kernel, at $\approx 56 T^2 C$ flop per head over the B batch and H heads, so its intensity rises with T .

3. OPTIMIZATION PERFORMED

We apply the same four-stage ladder to every operator. `v0` is a naive C++ baseline with dense scalar loops and no algebraic knowledge. `v1` exploits the fixed sparsity, sign patterns, and equivariant identities to cut the operation count. `v2` keeps that count but restructures for locality, mainly register-resident accumulators, straight-line per-blade kernels, and flash-style tiling for attention. `v3` adds explicit single-instruction-multiple-data (SIMD) vectorization with fused multiply-add (FMA). On the Tiger Lake target it uses the 512-bit Advanced Vector Extensions (AVX-512), 16 float lanes per instruction, and every kernel keeps a 256-bit AVX2 path of the same scheme so the extension still builds and runs where AVX-512 is absent. We then profiled the assembled model and focused the deepest optimization on its two bottlenecks, the equivariant linear map and the geometric attention. Every version is verified against the EzGATr PyTorch reference to floating-point tolerance.

Bilinears: sparsity and straight-line code. The geometric product and join improve in three stages over the dense `v0` baseline, which runs the full triple loop of 4096 MACs per multivector with nearly all of them multiplying by zero. First, `v1` keeps only the 192 (product) and 81 (join) nonzero (i, j, k, sign) entries of Γ and loops over

those, skipping the multiplications by zero. Second, `v2` unrolls this into one straight-line expression per output blade with the signs folded into constants, as in

$$o_4 = x_0y_4 + x_2y_9 + \dots - x_{14}y_8.$$

The 16 outputs now stay in registers with a single store each, and the independent blade expressions keep the out-of-order core busy. Extra loop unrolling and per-blade accumulators on top of `v2` gave input-dependent marginal gains, since the out-of-order core already overlaps the independent blades. Third, `v3` vectorizes across multivectors, in a structure-of-arrays (SoA) layout so the same blade from 8 consecutive multivectors sits in one 256-bit AVX2 vector and each per-blade expression runs on 8 at once, with a scalar remainder for the last $M \bmod 8$ and two accumulators on blades of four or more terms. Our first `v3` staged the SoA operands through memory; on our development machine (Raptor Lake, AVX2 only) this beat the auto-vectorized `v2`, but on the Tiger Lake target it did not, because Tiger Lake’s single 512-bit FMA port gives AVX-512 no throughput over two 256-bit AVX2 issues and the auto-vectorizer keeps its SoA operands in registers rather than paying that memory round trip. We therefore rebuilt `v3` to match the target: register-resident 256-bit AVX2 that holds every SoA operand in registers with no stack round trip, which is the version that wins on the target. The join adds one further, kernel-independent win. It needs only blade 14 of the reference, so the wrapper reads those M scalars rather than broadcasting the full $M \times 16$ reference. Finally, in the model’s bilinear layer the geometric product and join are fused into a single pass over the channels, so their shared inputs are read once and no intermediate multivectors are materialized, which yields a $2\text{--}6\times$ speedup for the bilinear layer.

equi_linear: sparse couplings and cached weights.

`equi_linear` improves in three stages over the dense `v0` baseline. For each (output, input) channel pair, `v0` multiplies the input multivector by a full 16×16 blade matrix (256 multiply-adds), although only 24 of its entries are nonzero. First, `v1` evaluates only those 24 couplings. Second, `v2` restructures for locality. It hoists the weight normalization out of the batch loop, since the inverse-norm factor depends only on the channel pair and not on the batch element, turning $B \times C^2 \times 9$ multiplications into $C^2 \times 9$, and it accumulates each output multivector in 16 registers with a single store instead of a read-modify-write per input channel, cutting the writes per (b, o) by about $96\times$. Third, `v3` vectorizes. The whole 16-blade multivector occupies one 16-lane register, so the 24 couplings of a channel pair collapse to just 2 FMAs per input channel. The packed diagonal weights provide the input for one FMA, while the input lanes for the other are selected from the off-diagonal sources by a single `_mm512_permutexvar_ps`. The kernel blocks 4 output channels by 2 batch rows, which keeps

eight independent accumulators in flight to hide the FMA latency. Beyond the kernel, two changes cut memory and allocation overhead. First, both `v2` and `v3` cache their prepared weights process-wide, keyed by the weight tensor, so `v2`’s normalization and `v3`’s weight packing run once per run rather than once per call. Second, the qkv projection output exceeds the last-level cache, so `v3` writes it with non-temporal stores (`_mm512_stream_ps`) that skip the read-for-ownership of the destination, with the bias folded into the accumulator so no second pass touches it.

equi_rms_norm and gated GELU. Both operators are memory-bound, so the gains come from cutting passes over the data and then widening each pass rather than from arithmetic throughput. RMS-norm improves in four stages. `v0` is a dense C++ scalar loop that forms the squared norm over all 16 blades through a full metric diagonal, half of whose entries are zero, then takes a second pass to apply the per-group scale. `v1` exploits the metric sparsity and sums only the 8 active blades, halving the reduction. `v2` keeps that operation count but unrolls the reduction across four independent accumulators to break the dependency chain and specializes the optional weight at compile time. `v3` vectorizes that kernel. It loads all 16 blades of a row in a single 512-bit `vmovups` and applies a precomputed 16-lane mask to zero the metric-inactive blades, then accumulates $\langle x, x \rangle$ with four independent FMA accumulators to break the reduction dependency chain. The reciprocal square root of the mean uses a scalar `rsqrtss` and one Newton-Raphson step, accurate to about 23 bits, replacing a full square root and divide. The gated GELU follows the same path. `v0` is a dense C++ scalar loop. `v1` computes the shared gate a single time per multivector and reuses it across all 16 blades, cutting the transcendental work by $16\times$, and `v2` streams the multivectors through a tight contiguous kernel. `v3` amortizes the transcendental across 16 multivectors per iteration under AVX-512 via a stride-16 gather `_mm512_i32gather_ps`, a vectorized tanh from a Cephes-style polynomial, and a single 512-bit broadcast-multiply per multivector.

Geom. attention: fused, cache-resident flash SDPA.

Attention has two costs, the IPA and DAA feature assembly and the $O(T^2)$ SDPA. `v0` builds the features with an `einsum` and runs a scalar SDPA that materializes the full $T \times T$ score matrix. `v1` replaces the `einsum` with the explicit DAA formula and caches the constant IPA selector and DAA basis tensors. `v2` replaces the materialized SDPA with a flash-attention core that tiles the queries and keys and maintains the softmax online through a running max and sum, so the scores never leave cache. When a later tile raises the running maximum, the partial output is rescaled by the correction factor, so one pass over the keys produces the full softmax without ever storing a score row. `v3` vectorizes the flash inner kernel with AVX-512 and FMA. The

$Q \cdot K^\top$ micro-kernel is a 4×32 register block, where each of four query rows is broadcast against two 16-lane key vectors to fill eight accumulators, and the $P \cdot V$ update is a 4×16 register block, both with masked 16-wide tails so no scalar remainder is needed. The softmax exponentials use a vectorized polynomial and the per-row score maximum a vectorized reduction. Each key tile is packed into the transposed layout the $Q \cdot K^\top$ kernel consumes once per head and reused across all query blocks, so the per-query-block key repack disappears. The feature assembly is folded into the flash loop. Each head’s $12C$ -wide query and key rows are written into thread-local scratch that stays resident, and the SDPA runs directly on it, removing the global assemble-then-reload round trip, and the head output is written in the $(B, T, H, C, 16)$ order the model’s post-attention reshape expects, making that reshape a zero-copy view, not a full copy per layer.

Beyond the kernel itself, the fused path has to fit the model’s tensors and access pattern. It reads the learned attention weights as a per-head per-channel table and follows the strides of the non-contiguous query, key, and value views, needing only each $(C, 16)$ multivector block to be contiguous, so nothing has to be copied into a dense buffer first. The default causal mask is applied inside the online softmax by skipping the key tiles that lie fully above the diagonal and masking the single diagonal tile, which also halves the tile work. Finally, the value rows sit 24 KB apart in the model’s strided view, a stride the hardware prefetcher does not follow, so the $P \cdot V$ kernel issues a single software prefetch of an upcoming value row to hide that latency. Together these make the fused, cache-resident flash core the dominant attention win, and the flash core is the reason the end-to-end advantage grows with T .

4. EXPERIMENTAL RESULTS

We evaluate the four-stage ladder on a single core, reporting speedup over the naive $v0$ per kernel and placing each kernel on the roofline.

Experimental setup. All measurements run on one thread of an Intel Core i7-1165G7 (Tiger Lake, 4 cores, base 2.8 GHz), with per-core caches L1D 48 KB, L2 1.25 MB, and a shared L3 of 12 MB. The single-core single-precision peak is 32 flop per cycle, or 89.6 GFLOP/s at 2.8 GHz, reached as 16-wide AVX-512 over one fused 512-bit FMA port or 8-wide AVX2 over two 256-bit FMA ports. We use a measured streaming bandwidth of 45 GB/s, which puts the roofline ridge at about 2.0 flop per byte. The extension is built with g++ at -O3 -march=native, which enables -mfma, -mavx2, and -mavx512f. The $v2$ scalar baseline is built with the auto-vectorizer disabled (-fno-tree-vectorize), so the $v2$ to $v3$ gap reflects the full effect of hand vectorization, and Table 2 adds the auto-vectorized build to show

how much of that gap the compiler recovers on its own. Each point is the minimum over many adaptive repetitions after a warm-up window, with one kernel resident at a time. Performance is reported as FLOP/cycle ($\text{PAPI_FP_OPS} / \text{PAPI_TOT_CYC}$) multiplied by the 2.8 GHz base frequency. Inputs are random multivector tensors with $B=2n$, $H=4n$, $T=128n$, and $C=8$, sweeping n from 1 to 12, and 1 to 9 for the $O(T^2)$ attention. We hold the channel width $C=8$ fixed since channel width is a design-time architectural constant during deployment, and this setup kept out per-point runtime at practical levels. Rooflines are read at $n=3$ (working set in cache) and $n=9$ (spilled to DRAM), and intensities are measured with PAPI.

Metric. We report speedup over $v0$ as the primary cross-version metric. Speedup is the meaningful comparison because the $v0$ to $v1$ step changes the useful-flop count of the bilinear and linear kernels by about 11 to 51 times, as the dense baseline works over zero-padded blades that the sparse versions skip, so a GFLOP/s ratio spanning $v0$ would compare different amounts of work. The same caveat governs the rooflines. A kernel’s versions belong on one roofline only when they share the work count, so the attention roofline shows all of $v0$ to $v3$, whose work is stable and whose intensity rises only because fusion cuts memory traffic, while the equi-linear roofline shows only the sparse $v1$ to $v3$ and excludes the dense $v0$ and the PyTorch reference, which compute a different (dense) map. Comparisons that cross that boundary, in particular against PyTorch, are reported as runtime speedups in Table 1.

Cost and speedup are two different rankings. Two kernels matter for different reasons. Geometric attention is the execution bottleneck, since it is the only $O(T^2)$ kernel and dominates runtime once the sequence is long, so its rewrite moves the full-model time the most. The largest per-kernel speedup, however, goes to the geometric product at $346\times$ (Table 1), because its dense $v0$ ran the full 4096-entry triple loop with only about five percent nonzero, so removing that waste and then restructuring for locality compounds to the biggest ratio, even though explicit SIMD adds little on top. The equivariant linear map sits between the two, carrying most of the non-attention compute because its cost is the only one quadratic in the channel count, and it is the kernel where explicit SIMD pays the most.

Geometric attention. Attention is the execution bottleneck but not the largest speedup. The sparsity and locality steps help little here, and the win is the fused, cache-resident flash SDPA at $v3$, a $v3/v0$ of $67\times$ (Table 1). Fig. 1 shows why on the roofline at $n=3$. Attention is stable-W, so all four versions do the same work and belong on one roofline. Every version sits right of the 2.0 flop/byte ridge, so attention is compute-bound in principle, yet $v0$, $v1$, and $v2$ sit at 0.8, 0.8, and 4.1 GFLOP/s, far below the 89.6 GFLOP/s ceiling, because the materialized $T \times T$

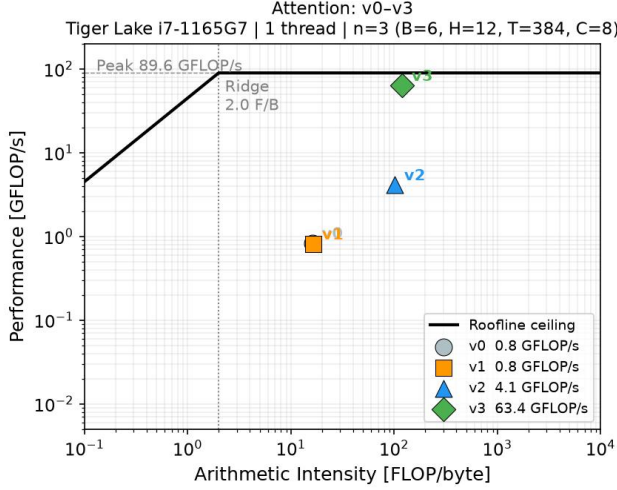


Fig. 1. Roofline for geometric attention at $n=3$. All four versions do the same work, but only the fused v_3 reaches the roof at 63.4 GFLOP/s (70.8% of peak). v_0 0.8, v_1 0.8, v_2 4.1 GFLOP/s.

scores and the assemble-then-reload feature round trip leave the FMA units idle. The fused v_3 reaches 63.4 GFLOP/s, 70.8% of peak and a $15.4\times$ step over v_2 , and its intensity climbs from about 16 to over 100 flop/byte as fusion turns memory traffic into in-cache reuse. Being right of the ridge only marks a kernel as potentially compute-bound. v_0 to v_2 are limited by overhead and round trips, and only the fused v_3 converts the available intensity into throughput. Against the reference, PyTorch reaches only 9.2 GFLOP/s on the same work, so v_3 is $6.9\times$ faster.

Equivariant linear map. The linear map carries most of the non-attention compute and is where vector width pays, climbing through the SIMD step rather than flattening, for a v_3/v_0 of $112\times$ (Table 1). Fig. 2 places v_1 to v_3 on the roofline at $n=3$. We leave v_0 out on purpose. It computes the dense 256-MAC map and so lands near intensity 1000 flop/byte, far from the sparse v_1 to v_3 near 250 to 350, and putting the two on one GFLOP/s axis would compare two different amounts of work. This is the W-shift, and the reason we measure the v_0 gain as a runtime ratio rather than on the roofline. Among the same-work sparse versions, v_1 sits at 3.0 GFLOP/s, v_2 climbs to 13.8, and v_3 reaches 53.8 GFLOP/s, 60.1% of peak, once a whole multivector fits in one register. All three lie far right of the ridge, so the kernel is compute-bound and the gap to the roof is implementation efficiency, closed only at v_3 . PyTorch runs the same dense map as v_0 , using the hardware well but spending most of its flops on the structural zeros our kernel skips, so in runtime, the fair comparison, v_3 is $9.7\times$ faster.

Per-kernel summary. Table 1 collects the per-kernel results at $n=3$. Two readings matter. First, the speedup column is set by how wasteful each dense v_0 was: the

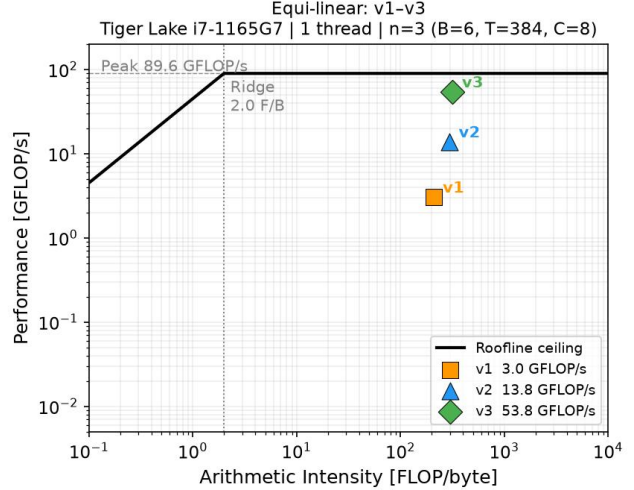


Fig. 2. Roofline for the equivariant linear map at $n=3$, same-work sparse v_1 to v_3 . The dense v_0 and PyTorch are excluded as different-work. v_1 3.0, v_2 13.8, v_3 53.8 GFLOP/s (60.1% of peak).

geometric product leads at $346\times$ with the join close behind at $315\times$, both bilinears whose dense baseline was almost all multiplies by zero, the linear map follows at $112\times$ and attention at $67\times$, and the two normalizations gain the least because their v_0 already does only a few flops per blade. Second, at $n=3$ the working set is cache-resident, so PAPI places every kernel in the compute region, yet only the equivariant linear map and attention are the closest to the FMA roof, at 60.1% and 70.8% of peak respectively. The other four sit well below for kernel-specific reasons rather than DRAM bandwidth at this size. The geometric product at 31.7% and the sparser join at 14.2% are held back by the structure-of-arrays shuffle traffic that surrounds their few arithmetic ops, RMS-norm at 11.4% by its short masked reduction, and gated GELU at 6.5% by the permultivector $\tanh$. The PyTorch column confirms the practitioner story. The gain is largest exactly where PyTorch dispatches a generic `einsum` over a tiny irregular contraction, $69.5\times$ on the geometric product and $64.9\times$ on the join, and smallest where PyTorch already has a vectorized path, $2.2\times$ on gated GELU.

The memory wall. Fig. 3 sweeps the multivector count $M=B T$ and plots v_3 performance for the three largest kernels, with the L2 and L3 capacities marked. Attention holds near peak across the whole sweep, about 60 to 67 GFLOP/s, and never rolls off, because its intensity rises with T and it stays arithmetic-limited at every size. The equivariant linear map climbs to about 56 GFLOP/s while its activation fits in cache, then rolls off once M pushes the working set past L3, falling to about 42 GFLOP/s at the largest size. That roll-off is the memory wall. The linear map is compute-bound only while cache-resident, and at scale its qkv ac-

Kernel	v_3/v_0	GF/s	%pk	bound	vs Py
geom. prod.	346×	28.4	31.7	comp	69.5×
equi. join	315×	12.7	14.2	comp	64.9×
equi. linear	112×	53.8	60.1	comp	9.7×
attention	67×	63.4	70.8	comp	6.9×
RMS norm	16×	10.3	11.4	comp	20.6×
gated GELU	2.5×	5.8	6.5	comp	2.2×

Table 1. Per-kernel results at $n=3$ (PAPI). Speedup is v_3 over v_0 , GF/s and %pk are the v_3 values, bound is the measured region, and “vs Py” is the runtime speedup over the EzGATr PyTorch reference.

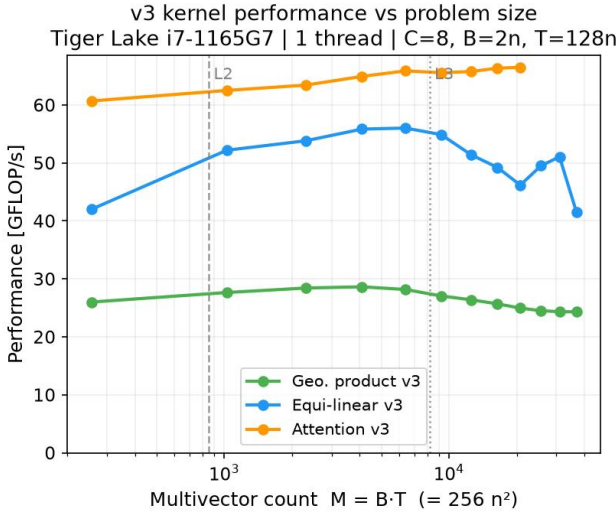


Fig. 3. v_3 performance versus multivector count $M=BT$, L2 and L3 marked. Attention holds near peak, the geometric product stays at its structural ceiling, and the linear map rolls off past L3 as its activation spills to DRAM.

tivation streams from DRAM and the 45 GB/s bandwidth caps it. The geometric product sits at its structural ceiling near 24 to 29 GFLOP/s throughout, held there by its shuffle traffic rather than by capacity, so it barely moves. This size-dependence is why the linear map gives up part of its 60.1% at large n .

End to end. Fig. 4 composes the kernels into the full two-layer model at one thread and reports the speedup over v_0 . The naive v_0 is fast enough to time directly only up to $n=3$ (solid), beyond which its runtime is extrapolated (dashed), so the large- n values are a projection rather than a measurement. The model is fastest at the smallest size, $38\times$ at $n=1$, falling through $20\times$ at $n=2$ to $15\times$ at $n=3$ and trending toward a plateau near $13\times$. The decline follows from how the two kinds of kernel scale. With $B=2n$, $H=4n$, and $T=128n$, attention costs $O(n^4)$ while every per-token kernel, the linear maps, the two normalizations,

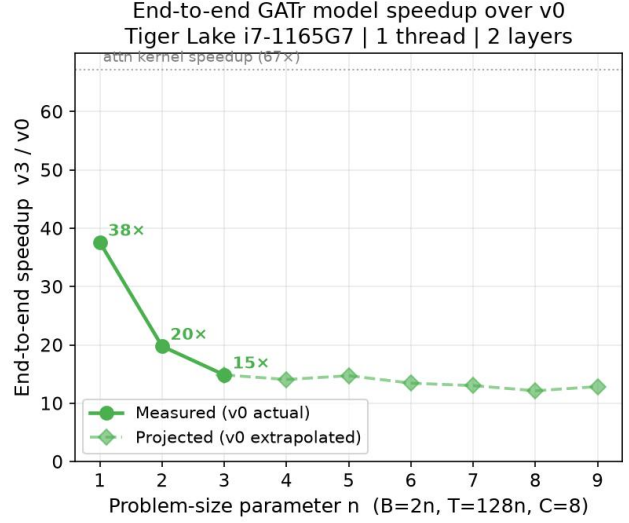


Fig. 4. End-to-end model speedup over v_0 versus n , single thread. v_0 is measured to $n=3$ (solid) and extrapolated beyond (dashed), so the plateau is a projection. The model peaks at $38\times$ ($n=1$) and trends toward roughly $13\times$. The dotted line marks attention’s isolated $67\times$, which the full model stays well below.

and the bilinears, costs $O(M)=O(n^2)$. At $n=1$ the per-token kernels carry the runtime, and because their dense v_0 baselines were the most wasteful, the heavily optimized linear map and bilinears lift the model to $38\times$. As n grows, attention’s $O(n^4)$ cost takes over. The measured v_3 runtime already grows by roughly the n^4 factor between sizes, so by $n=3$ the model is attention-dominated, and the end-to-end speedup converges toward what attention contributes in context, with the linear map giving up part of its gain once it spills from cache (Fig. 3). That in-model contribution is far below the $67\times$ attention reaches in isolation. Already at the measured $n=3$ the composite is only $15\times$, so the dotted $67\times$ line, an isolated-kernel number, sits well above the curve and is not the value the model approaches. A strong isolated-kernel speedup does not transfer intact once that kernel runs inside the model. Beyond $n=3$ the v_0 runtime is extrapolated with an n^4 term that the data constrains only to $n=3$, so we read the $13\times$ plateau as indicative. Speedup here is a runtime ratio, so v_0 belongs on this plot even for the W-shift kernels, unlike the GFLOP/s rooflines.

Hand SIMD versus the auto-vectorizer. Table 2 compares, per kernel at $n=3$, the auto-vectorized build and the hand-written v_3 , both as speedup over the no-vector scalar build. The pattern is the one the cost model implies. Where the kernel is not lifted by vector width the auto-vectorizer already does most of the job and hand SIMD pulls only modestly ahead, the geometric product at $5.29\times$ against the auto-vectorizer’s $4.6\times$, RMS-norm at $2.5\times$ against $2.2\times$, and the join at $2.73\times$ against $2.6\times$. The join’s margin is the

Kernel	auto-vec	hand $\vee 3$
geom. prod.	4.6 $\times$	5.29 $\times$
equi. join	2.6 $\times$	2.73 $\times$
equi. linear	1.5 $\times$	6.0 $\times$
attention	4.5 $\times$	54 $\times$
RMS norm	2.2 $\times$	2.5 $\times$
gated GELU	1.0 $\times$	3.7 $\times$
end to end	1.4 $\times$	

Table 2. Hand SIMD vs. the auto-vectorizer at $n=3$, speedup over the no-vector scalar build. The last row is the model-level gain of $\vee 3$ over auto-vec $\vee 2$.

slimmest because the compiler vectorizes its loop well and the join’s real win is algorithmic, the 315 $\times$ over $\vee 0$, not the SIMD step. Hand SIMD pulls clearly ahead where the kernel is compute-bound or needs fusion, the linear map from 1.5 $\times$ to 6.0 $\times$, the gather-bound gated GELU from 1.0 $\times$, where the compiler vectorizes nothing, to 3.7 $\times$, and above all attention from 4.5 $\times$ to 54 $\times$, because the compiler cannot fuse the flash loop or pack the key tiles. End to end, hand SIMD adds 1.4 $\times$ over the auto-vectorized build, so most of the model-level win is already algorithmic and locality and SIMD is the final layer.

Across compilers. Where it counts, the hand-written $\vee 3$ is compiler-independent. The full model runs in 156 ms under both GCC and Clang, because $\vee 3$ issues explicit AVX-512 and AVX2 intrinsics throughout and leaves the compiler nothing to vectorize. The compiler only matters for the builds that lean on auto-vectorization. The auto-vectorized $\vee 2$ model is 16% slower under Clang (254.9 ms against 220.2 ms), because Clang barely vectorizes the geometric product, the equivariant join, and RMS-norm and runs 5 to 6 $\times$ slower than GCC on them, while on the linear map it does the reverse and is about 20% faster. The naive $\vee 0$ gap is wider still, with GCC about 1.9 $\times$ faster, for the same reason. Among the $\vee 3$ kernels the large ones agree to within a few percent (the linear map, RMS-norm, and attention within 4%), and the sub-0.1 ms kernels show more spread, up to 28% on the geometric product, only because their absolute times are tiny and overhead-dominated, which is why the 156 ms model total is unaffected. This is the strongest form of the hand-SIMD argument. The auto-vectorized path is at the mercy of which compiler recognizes which pattern and its end-to-end result swings by compiler, whereas the explicit $\vee 3$ lands at the same time either way.

Why more is hard. At $n=3$ the two closest-to-peak kernels are the equivariant linear map at 60.1% and attention at 70.8% of 89.6 GFLOP/s. Their residual gap is shuffle traffic for the off-diagonal couplings and the short $C=8$ reduction, which a wider instruction set cannot remove. Attention holds this at every size, but the linear map keeps it only while cache-resident and rolls off to the bandwidth

roof once M spills (Fig. 3). The geometric product is held to about 31% of peak by its structure-of-arrays shuffles, which is why hand SIMD barely beats the auto-vectorizer on it, and the two normalizations are capped by a short reduction and a transcendental. In the bandwidth-bound regime the only lever is fewer bytes moved, already spent on single stores, masked in-register reductions, and the fused attention path. This sets the model’s two regimes: at small sizes the linear maps dominate, applied in every block, and as T grows the $O(T^2)$ attention takes over. Further gains therefore have to change the computation, e.g. using mathematical tricks or lowering precision, not tighten these kernels in place.

5. CONCLUSIONS

We rebuilt the performance-critical operators of a projective geometric algebra transformer as a four-stage ladder of single-core C++ kernels, from a naive scalar baseline to hand-written AVX-512 with an AVX2 fallback. A cost model predicted which kernels would saturate the FMA roof and which would be memory-limited, and the rooflines and the memory-scaling sweep confirmed it. Three results stand out. First, the largest speedups are algorithmic: the sparsest kernels gain the most, the geometric product leading at 346 $\times$, and most of the model-level win is in place before any SIMD, which adds only 1.4 $\times$ end to end over the auto-vectorized build. Composed into the full model the speedup is 38 $\times$ at the smallest size and trends toward roughly 13 $\times$ as the $O(T^2)$ attention comes to dominate and the linear map spills from cache. Second, explicit SIMD pays most on the compute-bound kernels, the linear map at 60.1% and the fused flash attention at 70.8% of single-core peak at $n=3$, while on the structure- and bandwidth-limited kernels it only ties the auto-vectorizer. Third, the vectorization that pays must be written by hand: the compiler cannot fuse the flash loop or pack its key tiles and does not vectorize the gated GELU at all, and the auto-vectorized model swings 16% between GCC and Clang, whereas the explicit $\vee 3$ runs in 156 ms under both. Against PyTorch our kernels win on every operator in runtime, from 69.5 $\times$ on the geometric product to 6.9 $\times$ on attention and 2.2 $\times$ on the gated GELU, the linear-map GFLOP/s looking close only because PyTorch spends most of its flops on the structural zeros we skip. The clearest paths beyond this are orthogonal to the kernels: multi-threading across the batch, head, and token dimensions, or `bfloat16` storage to halve the bytes the bandwidth-limited kernels move.

6. REFERENCES

- [1] Johann Brehmer, Pim de Haan, Sönke Behrends, and Taco Cohen, “Geometric algebra transformer,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, vol. 36, pp. 35472–35496, arXiv:2305.18415.
- [2] Yuxuan Wang and Xiatao Sun, “EzGATr: Easy geometric algebra transformer,” <https://github.com/Guest400123064/ezgatr>, 2024.
- [3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, vol. 35, arXiv:2205.14135.

7. CONTRIBUTIONS OF TEAM MEMBERS

Anastasija Tortevska. Owned the geometric attention kernel across all stages: the scalar materialized-SDPA v0, the explicit-DAA v1 with cached IPA/DAA selectors, the online-softmax flash core v2, and the AVX-512 v3 with the 4×32 $Q \cdot K^\top$ and 4×16 $P \cdot V$ register blocks. Built the fused, cache-resident flash SDPA that assembles the IPA/DAA features in-place and writes the head output in the model’s reshape order. Produced the roofline, PAPI-intensity, and memory-scaling analyses.

Kristijan Zafirovski. Owned the equivariant linear map across the ladder: the true-dense v0, the 24-coupling sparse v1, the locality-restructured v2 with hoisted weight normalization and single-store accumulation, and the AVX-512 v3 with the `permutexvar` off-diagonal gather, four-output-by-two-batch blocking, process-wide weight caches, and non-temporal qkv stores. Brought the gated GELU and RMS-norm to their vectorized v3 form and contributed a fused cache-resident AVX2 flash-SDPA attention variant.

Glenn Schönbächler. Wrote the baseline C++ implementations of the inner product, the equivariant RMS norm, and the scalar-gated GELU, and their early sparsity and locality optimizations (single shared gate, metric-active reduction) that seed the v1/v2 stages of the memory-bound kernels. Brought the gated GELU and RMS-norm to their vectorized v3 form. Experimented with SoA versions for equivariant RMS norm.

Mukhammadali Sayfiddinov. Ported the EzGATr operators to a C++ baseline and owned the two bilinear kernels, the geometric product and the equivariant join, across the full ladder: the dense v0, the runtime-sparse Γ -table v1, the straight-line per-blade v2 with the experiments on loop unrolling and inter-blade accumulators, and the SoA AVX2/AVX-512 v3, including the register-resident rebuild

and the Tiger-Lake transpose fix. Folded the join’s blade-14 reference read and fused the geometric product and join into a single bilinear pass. Also contributed the flash-SDPA attention variant that packs the keys once per head with guarded AVX-512 micro-kernels (merged into the final v3).

8. CODE

Fastest scalar kernel: the `equi_linear v2` inner loop. Weights are pre-normalized once (w_n , outside this loop), and a 16-wide accumulator stays in registers across all input channels and is stored once per (b, o) .

```
1  for (int64_t b = 0; b < batch; ++b) {
2      const T* __restrict__ xb = xp_base + b*in_ch*16;
3      for (int64_t o = 0; o < out_ch; ++o) {
4          T acc[16];
5          acc[0] = bp ? bp[o] : T(0);          // bias folded in
6          for (int k = 1; k < 16; ++k) acc[k] = T(0);
7          for (int64_t i = 0; i < in_ch; ++i) {
8              const T* __restrict__ w = wn + (o*in_ch + i)*9;
9              const T* __restrict__ xi = xb + i*16;
10             const T w0=w[0],w1=w[1],w2=w[2],w3=w[3],w4=w[4];
11             const T w5=w[5],w6=w[6],w7=w[7],w8=w[8];
12             // grade-block diagonal (16 couplings)
13             acc[0] += w0*xi[0];
14             for (int k=1; k<5; ++k) acc[k] += w1*xi[k];
15             for (int k=5; k<11; ++k) acc[k] += w2*xi[k];
16             for (int k=11; k<15; ++k) acc[k] += w3*xi[k];
17             acc[15] += w4*xi[15];
18             // sparse e0-couplings (8 off-diagonal)
19             acc[1] += w5*xi[0];
20             acc[5] += w6*xi[2]; acc[6] += w6*xi[3];
21             acc[7] += w6*xi[4]; acc[11] += w7*xi[8];
22             acc[12] += w7*xi[9]; acc[13] += w7*xi[10];
23             acc[15] += w8*xi[14];
24         }
25         T* __restrict__ op = op_base + (b*out_ch + o)*16;
26         for (int k = 0; k < 16; ++k) op[k] = acc[k];
27     }
28 }
```

Fig. 5. Grade-block diagonal convolution kernel

Fastest SIMD kernel, the `equi_linear v3 float32` inner loop on the AVX-512 target. The whole 16-blade multivector is one `_m512`, so the 24 scalar couplings become 2 FMAs per input channel, one diagonal and one `permutexvar`-gathered off-diagonal. The shipped kernel blocks four output channels by two batch rows. The single-accumulator body is shown for clarity, and a build without AVX-512 runs the same map as a two-half AVX2 form.

```

1 // packed once per (o,i): W512[32] =
2 //   [diag_lo(8) | diag_hi(8) | off_lo(8) | off_hi(8)]
3 // combined off-diagonal permute (absolute lane indices):
4 const __m512i idx =
5     _mm512_setr_epi32(0,0,0,0,0,2,3,4, 8,8,8,8,9,10,8,14);
6 // bias folded into lane 0 of the accumulator
7 __m512 acc = bp ? _mm512_maskz_broadcastss_ps(0x0001u, _mm_set_ss(bp[o]))
8                 : _mm512_setzero_ps();
9 const float* wv = W512 + o*in_ch*32;
10 for (int64_t i = 0; i < in_ch; ++i) {
11     const __m512 x = _mm512_loadu_ps(xb + i*16); // whole 16-blade multivector
12     const float* w = wv + i*32;
13     // diagonal: 1 FMA
14     acc = _mm512_fmadd_ps(_mm512_loadu_ps(w), x, acc);
15     // off-diagonal: gather source lanes, then 1 FMA
16     acc = _mm512_fmadd_ps(_mm512_loadu_ps(w + 16),
17                           _mm512_permutexvar_ps(idx, x), acc);
18 }
19 _mm512_storeu_ps(op_base + (b*out_ch + o)*16, acc); // single store per output

```

Fig. 6. AVX-512 vectorized convolution kernel